Name: M.Divya

Reg. No: 192373046
DSA1604 – Knowledge Engineering and Data Science

Exercise 1: Entailment Checking in a Smart-Home Alarm (Propositional Logic)

Aim

To verify logical entailment (KB |= query) by model checking over all truth assignments, using a truth table.

Software Required

Python 3.x (IDLE / any Python IDE) and the itertools module

Algorithm

1. Start the program. List all propositional symbols: M (motion), A (armed), D (door open), S (siren).
2. Encode the knowledge base as the conjunction of R1: (M AND A) -> S, R2: D -> M and the facts A and D.
3. Generate all 2^n truth assignments with itertools.product.
4. Evaluate the KB and the query in every model and print one row of the truth table per model.
5. If any model makes the KB true and the query false, entailment fails; otherwise the KB entails the query.
6. Test the queries S and NOT S, print the result and stop the program.

Pseudocode

```
START
    symbols <- {M, A, D, S}
    FOR EACH query IN {S, NOT S}
        entails <- TRUE
        FOR EACH assignment OF symbols
            k <- KB(assignment)  (R1 AND R2 AND A AND D)
            q <- query(assignment)
            DISPLAY row of truth table
            IF k = TRUE AND q = FALSE THEN entails <- FALSE
        END FOR
        DISPLAY "KB entails query ?", entails
    END FOR
STOP
```

Program

```
import itertools

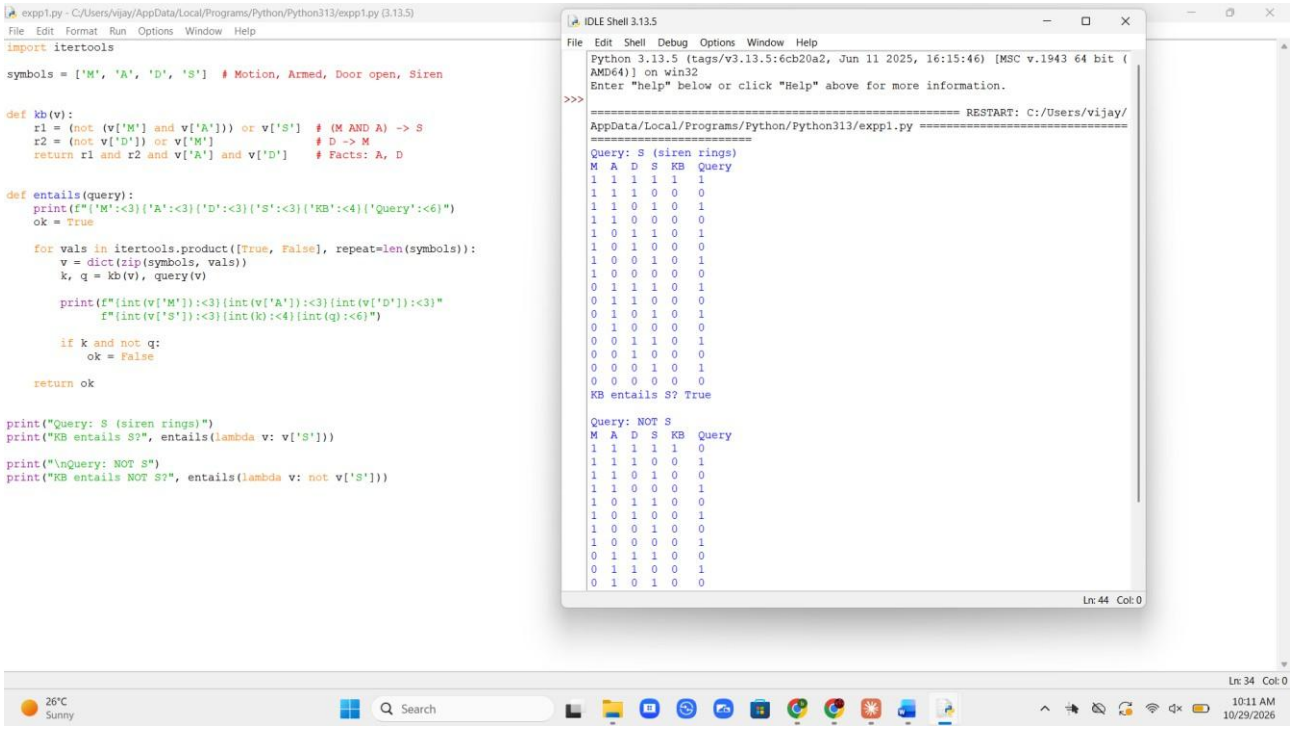
symbols = ['M', 'A', 'D', 'S']      # Motion, Armed, Door open, Siren

def kb(v):
    r1 = (not (v['M'] and v['A'])) or v['S']    # (M ^ A) -> S
    r2 = (not v['D']) or v['M']                 # D -> M
    return r1 and r2 and v['A'] and v['D']      # A, D are facts

def entails(query):
    print(f"{'M':<3}{'A':<3}{'D':<3}{'S':<3}{'KB':<4}{'Query':<6}")
    ok = True
    for vals in itertools.product([True, False], repeat=len(symbols)):
        v = dict(zip(symbols, vals))
        k, q = kb(v), query(v)
        print(f"{int(v['M']):<3}{int(v['A']):<3}{int(v['D']):<3}"
              f"{int(v['S']):<3}{int(k):<4}{int(q):<6}")
        if k and not q:
            ok = False
    return ok

print("Query: S (siren rings)")
print("KB entails S ?", entails(lambda v: v['S']))
print("\nQuery: not S")
print("KB entails not S ?", entails(lambda v: not v['S']))
```

Output Screenshot



Result

The only model satisfying the KB has S = True, so KB |= S and the siren will ring. The KB does not entail NOT S.

Exercise 2: Resolution Refutation for a Login System (Propositional Logic)

Aim

To prove a propositional query by converting the knowledge base to clause form and deriving the empty clause.

Software Required

Python 3.x (IDLE / any Python IDE), itertools.combinations and frozensets for clauses

Algorithm

1. Start the program. Convert the KB to CNF: (~P v Q), (~Q v R) and (P).
2. Negate the query R and add the clause (~R) to the clause set.
3. Pick every pair of clauses; if one contains literal L and the other contains ~L, produce the resolvent (union of both clauses without L and ~L).
4. Add every new resolvent to the clause set and print the resolution step.
5. If the empty clause appears, the query is proved. If no new clauses appear, the query is not provable.
6. Display the result and stop the program.

Pseudocode

```
START
    clauses <- {~P v Q}, {~Q v R}, {P}, {~R}
    REPEAT
        FOR EACH pair (c1, c2) IN clauses
            FOR EACH resolvent r OF (c1, c2)
                IF r IS NOT IN clauses THEN
                    DISPLAY step
                    ADD r TO new
                    IF r = empty clause THEN proved <- TRUE
                END IF
            END FOR
        END FOR
        clauses <- clauses + new
    UNTIL proved OR new IS EMPTY
    DISPLAY "Query R proved:", proved
STOP
```

Program

```
from itertools import combinations

def neg(l):
    return l[1:] if l.startswith('~') else '~' + l

def resolve(c1, c2):
    out = []
    for l in c1:
        if neg(l) in c2:
            out.append(frozenset((c1 - {l}) | (c2 - {neg(l)})))
    return out

def fmt(c):
    return '{' + ', '.join(sorted(c)) + '}' if c else '{} (empty)'

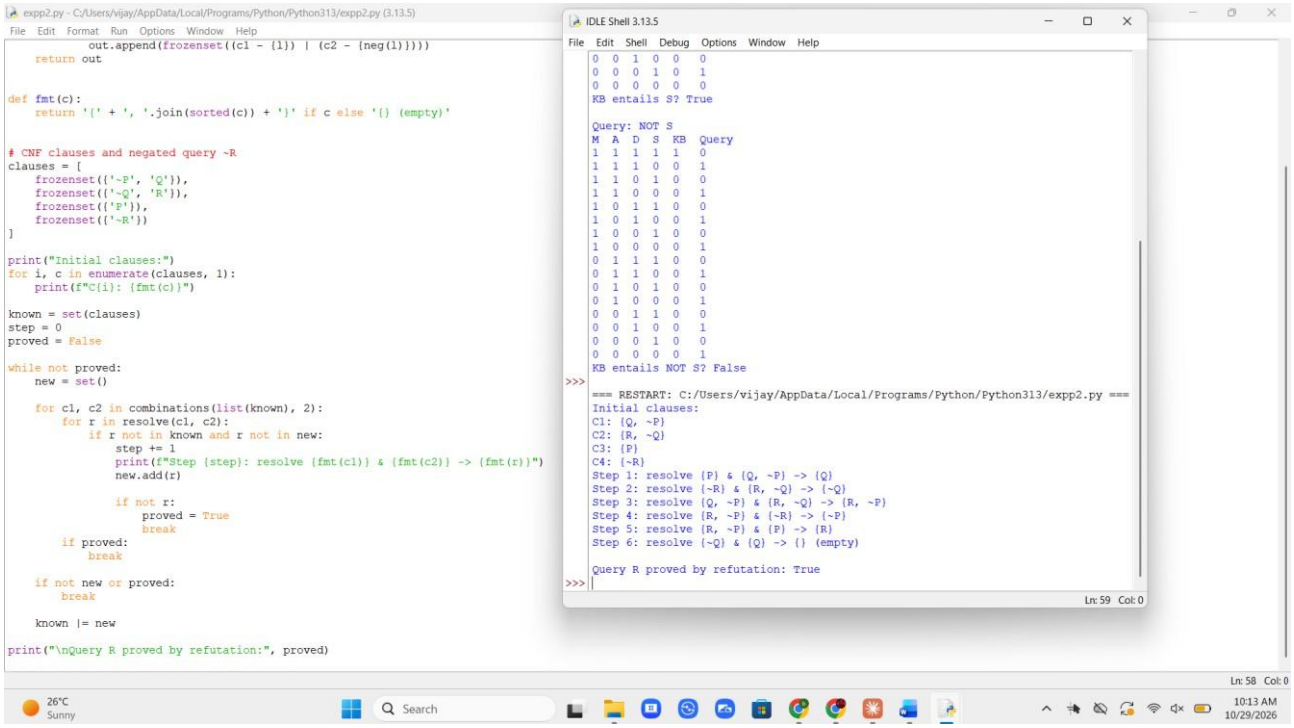
clauses = [frozenset({'~P', 'Q'}), frozenset({'~Q', 'R'}), frozenset({'P'}),
           frozenset({'~R'})]          # last clause = negated query

print("Initial clauses:")
for i, c in enumerate(clauses, 1):
    print(f"  C{i}: {fmt(c)}")

known = set(clauses)
step = 0
proved = False
while not proved:
    new = set()
    for c1, c2 in combinations(list(known), 2):
        for r in resolve(c1, c2):
            if r not in known and r not in new:
                step += 1
                print(f"Step {step}: resolve {fmt(c1)} & {fmt(c2)} -> {fmt(r)}")
                new.add(r)
                if not r:
                    proved = True
                    break
        if proved:
            break
    if proved or not new:
        break
    known |= new

print("\nQuery R proved by refutation:", proved)
```

Output Screenshot



Result

The empty clause was derived, so the KB together with ~R is unsatisfiable. Hence R (dashboard accessible) logically follows from the rules. The order of the steps may vary from run to run.

Exercise 3: Unification for Student-Course Queries (Predicate Logic)

Aim

To implement the unification algorithm that finds a substitution making two first-order terms identical.

Software Required

Python 3.x (IDLE / any Python IDE); tuples for predicates and compound terms, dictionaries for substitutions

Algorithm

1. Start the program. Treat names starting with an uppercase letter as variables and the rest as constants.
2. unify(x, y, s): if x equals y, return the current substitution s.
3. If x is a variable, call unify_var(x, y). If y is a variable, call unify_var(y, x).
4. If both are tuples of equal length, unify them element by element, passing the substitution along.
5. Otherwise return failure (None).
6. unify_var(v, t): if v is already bound, unify its binding with t; otherwise add the binding v -> t.
7. Run the function on five test pairs, print each substitution and stop.

Pseudocode

```
START
    FUNCTION unify(x, y, s)
        IF s = FAIL THEN RETURN FAIL
        IF x = y THEN RETURN s
        IF x IS VARIABLE THEN RETURN unify_var(x, y, s)
        IF y IS VARIABLE THEN RETURN unify_var(y, x, s)
        IF x AND y ARE TUPLES OF EQUAL LENGTH THEN
            FOR EACH (a, b) IN (x, y): s <- unify(a, b, s)
            RETURN s
        RETURN FAIL
    END FUNCTION
    FUNCTION unify_var(v, t, s)
        IF v IN s THEN RETURN unify(s[v], t, s)
        IF t IS VARIABLE AND t IN s THEN RETURN unify(v, s[t], s)
        RETURN s + {v -> t}
    END FUNCTION
    FOR EACH test pair (a, b): DISPLAY unify(a, b, {})
STOP
```

Program

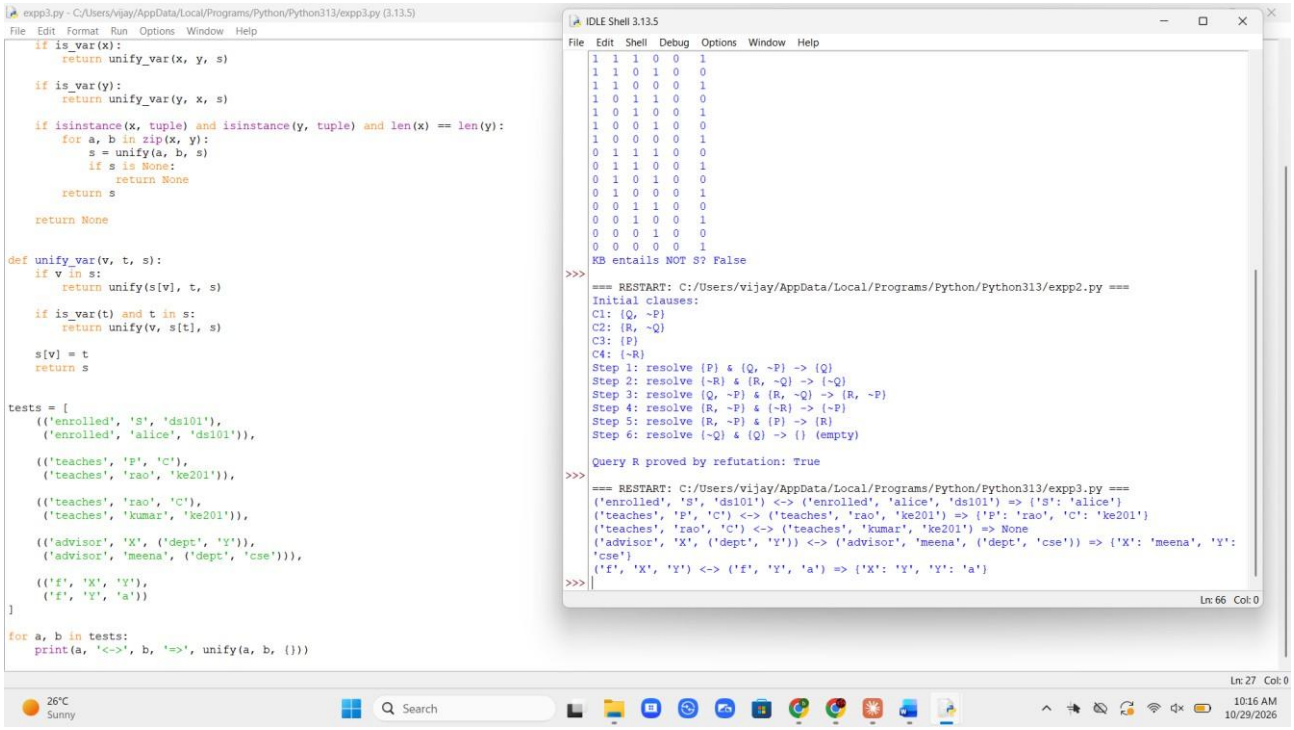
```
def is_var(x):
    return isinstance(x, str) and x[0].isupper()

def unify(x, y, s):
    if s is None:
        return None
    if x == y:
        return s
    if is_var(x):
        return unify_var(x, y, s)
    if is_var(y):
        return unify_var(y, x, s)
    if isinstance(x, tuple) and isinstance(y, tuple) and len(x) == len(y):
        for a, b in zip(x, y):
            s = unify(a, b, s)
        return s
    return None

def unify_var(v, t, s):
    if v in s:
        return unify(s[v], t, s)
    if is_var(t) and t in s:
        return unify(v, s[t], s)
    return {**s, v: t}

tests = [
    (('enrolled', 'S', 'ds101'), ('enrolled', 'alice', 'ds101')),
    (('teaches', 'P', 'C'), ('teaches', 'rao', 'ke201')),
    (('teaches', 'rao', 'C'), ('teaches', 'kumar', 'ke201')),
    (('advisor', 'X', ('dept', 'Y')), ('advisor', 'meena', ('dept', 'cse'))),
    (('f', 'X', 'Y'), ('f', 'Y', 'a')),
]
for a, b in tests:
    print(a, '<->', b, '=>', unify(a, b, {}))
```

Output Screenshot



Result

Unification correctly bound variables to constants (including nested terms) and failed (None) for the clash rao vs kumar. The last case shows the chained binding X -> Y -> a. This implementation omits the occurs-check for simplicity.

Exercise 4: Evaluating Quantified Statements over a Class (Predicate Logic)

Aim

To evaluate universally and existentially quantified formulas over a finite domain and to verify the quantifier duality NOT EXISTS NOT-P = FORALL P.

Software Required

Python 3.x (IDLE / any Python IDE); all() for the universal and any() for the existential quantifier

Algorithm

1. Start the program. Define the domain of discourse (the students) and the predicates as dictionaries and sets.
2. Translate FORALL into all(...) and EXISTS into any(...).
3. Translate the implication A -> B as (not A) or B.
4. Evaluate the statements S1 to S5 and print the truth value of each.
5. Compare S1 and S4 to check De Morgan's law for quantifiers.
6. Display the result and stop the program.

Pseudocode

```
START
    students <- {Arun, Bala, Chitra, Divya}
    DEFINE enrolled, attends, ds_marks
    S1 <- FORALL x: (Enrolled(x, KE) -> Attends(x))
    S2 <- EXISTS x: (Enrolled(x, DS) AND Marks(x) >= 90)
    S3 <- FORALL x EXISTS c: Enrolled(x, c)
    S4 <- NOT EXISTS x: (Enrolled(x, KE) AND NOT Attends(x))
    S5 <- FORALL x: (Enrolled(x, DS) -> Marks(x) >= 80)
    DISPLAY S1, S2, S3, S4, S5
    DISPLAY "Duality holds:", S1 = S4
STOP
```

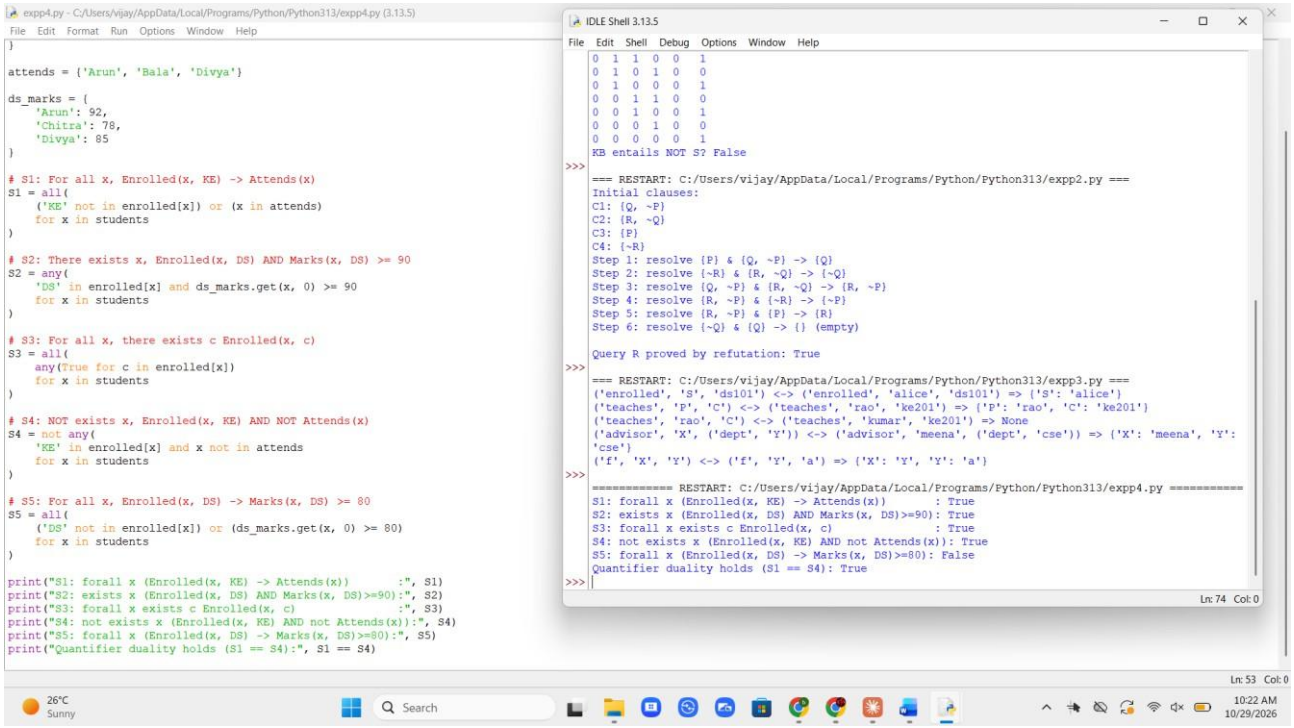
Program

```
students = ['Arun', 'Bala', 'Chitra', 'Divya']
enrolled = {'Arun': {'KE', 'DS'}, 'Bala': {'KE'},
            'Chitra': {'DS'}, 'Divya': {'KE', 'DS'}}
attends = {'Arun', 'Bala', 'Divya'}
ds_marks = {'Arun': 92, 'Chitra': 78, 'Divya': 85}

S1 = all((('KE' not in enrolled[x]) or (x in attends)) for x in students)
S2 = any(('DS' in enrolled[x] and ds_marks.get(x, 0) >= 90) for x in students)
S3 = all(any(True for c in enrolled[x]) for x in students)
S4 = not any(('KE' in enrolled[x] and x not in attends) for x in students)
S5 = all(ds_marks.get(x, 0) >= 80 for x in students if 'DS' in enrolled[x])

print("S1: forall x (Enrolled(x,KE) -> Attends(x))      :", S1)
print("S2: exists x (Enrolled(x,DS) ^ Marks(x,DS)>=90)  :", S2)
print("S3: forall x exists c Enrolled(x,c)              :", S3)
print("S4: not exists x (Enrolled(x,KE) ^ not Attends(x)):", S4)
print("S5: forall x (Enrolled(x,DS) -> Marks(x,DS)>=80)  :", S5)
print("Quantifier duality holds (S1 == S4):", S1 == S4)
```

Output Screenshot



Result

S1 to S4 are true; S5 is false because Chitra (DS marks 78) has marks below 80. S1 and S4 always agree, confirming the quantifier duality law.

Exercise 5: Modus Ponens and Modus Tollens in Network Fault Diagnosis (Inference Mechanisms)

Aim

To implement two basic inference rules: Modus Ponens (A, A->B therefore B) and Modus Tollens (NOT B, A->B therefore NOT A).

Software Required

Python 3.x (IDLE / any Python IDE); a dictionary of known truth values

Algorithm

1. Start the program. Store the rules as (antecedent, consequent) pairs: Power->RouterOn, RouterOn->WifiOn, WifiOn->InternetOK.
2. Store the known facts in a dictionary.
3. Repeat until nothing changes: for each rule A->B, if A is True and B is unknown, set B = True (Modus Ponens).
4. In the same pass, if B is False and A is unknown, set A = False (Modus Tollens).
5. Print every inference step as it is made.
6. Run Case A (Power = True) and Case B (InternetOK = False), print the final state and stop.

Pseudocode

```
START
    rules <- (Power, RouterOn), (RouterOn, WifiOn), (WifiOn, InternetOK)
    READ known facts
    REPEAT
        changed <- FALSE
        FOR EACH rule (a, c) IN rules
            IF a = TRUE AND c IS UNKNOWN THEN
                c <- TRUE, changed <- TRUE   (Modus Ponens)
            END IF
            IF c = FALSE AND a IS UNKNOWN THEN
                a <- FALSE, changed <- TRUE  (Modus Tollens)
            END IF
        END FOR
    UNTIL changed = FALSE
    DISPLAY final state
STOP
```

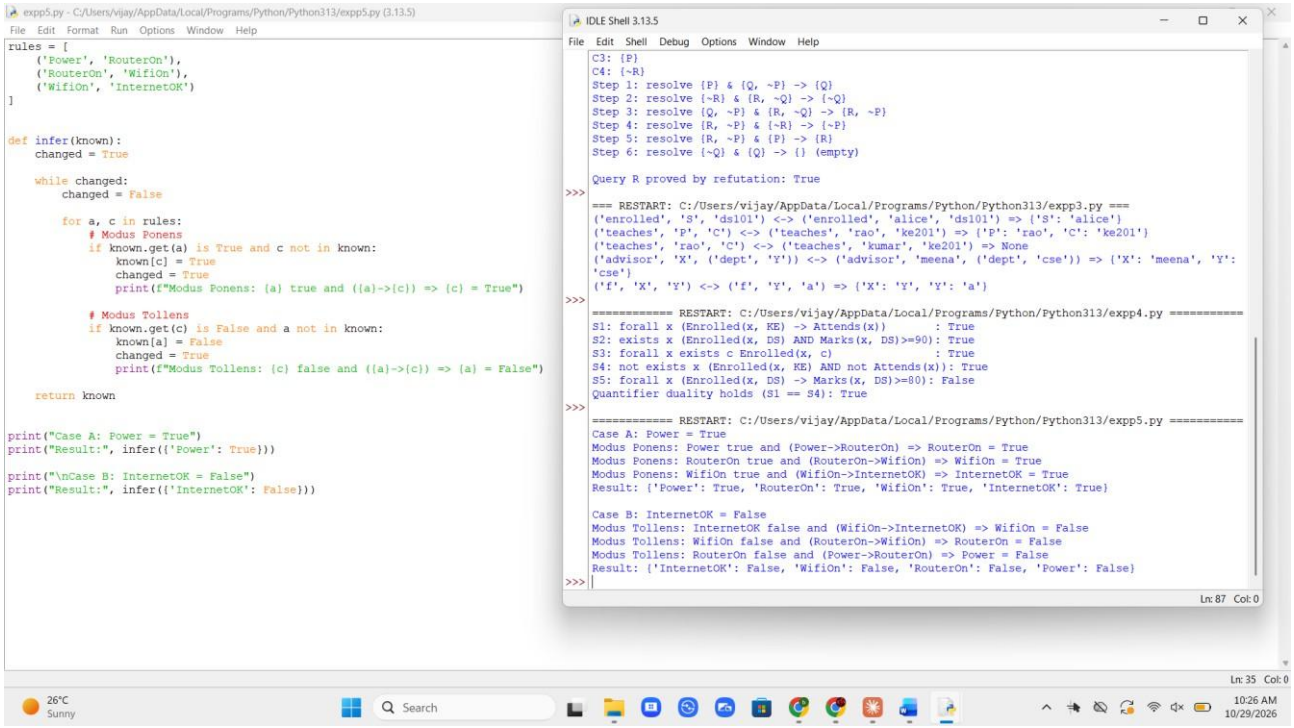
Program

```
rules = [('Power', 'RouterOn'), ('RouterOn', 'WifiOn'), ('WifiOn', 'InternetOK')]

def  infer(known):
    changed = True
    while changed:
        changed = False
        for a, c in rules:
            if known.get(a) is True and c not in known:
                known[c] = True
                changed = True
                print(f"Modus Ponens : {a} true and ({a}->{c}) => {c} = True")
            if known.get(c) is False and a not in known:
                known[a] = False
                changed = True
                print(f"Modus Tollens: {c} false and ({a}->{c}) => {a} = False")
    return known

print("Case A: Power = True")
print("Result:", infer({'Power': True}))
print("\nCase B: InternetOK = False")
print("Result:", infer({'InternetOK': False}))
```

Output Screenshot



Result

Modus Ponens propagated truth forward along the chain (power on implies internet OK). Modus Tollens propagated falsity backward (no internet implies Wi-Fi, router and power are all not working under this rule set).

Exercise 6: Loan Eligibility Using Forward Chaining (Forward and Backward Chaining)

Aim

To implement forward chaining (data-driven inference) in a production-rule system and decide loan approval.

Software Required

Python 3.x (IDLE / any Python IDE); sets for facts and premises

Algorithm

1. Start the program. Initialise working memory with the known facts: employed, salary_above_30k, credit_score_good, has_collateral.
2. Store the rules R1 to R4 as (set of premises, conclusion) pairs.
3. Scan the rules. If all premises of a rule are in the facts and its conclusion is not, fire the rule and add the conclusion.
4. Repeat the scan until no rule can fire (fixed point).
5. Check whether loan_approved is in the final fact set.
6. Display the final facts and the decision, then stop.

Pseudocode

```
START
    facts <- {employed, salary_above_30k, credit_score_good, has_collateral}
    rules <- R1, R2, R3, R4 (premises -> conclusion)
    REPEAT
        fired <- FALSE
        FOR EACH rule IN rules
            IF premises ARE IN facts AND conclusion IS NOT IN facts THEN
                ADD conclusion TO facts
                DISPLAY cycle, premises, conclusion
                fired <- TRUE
            END IF
        END FOR
    UNTIL fired = FALSE
    DISPLAY facts
    DISPLAY "Loan approved?", loan_approved IN facts
STOP
```

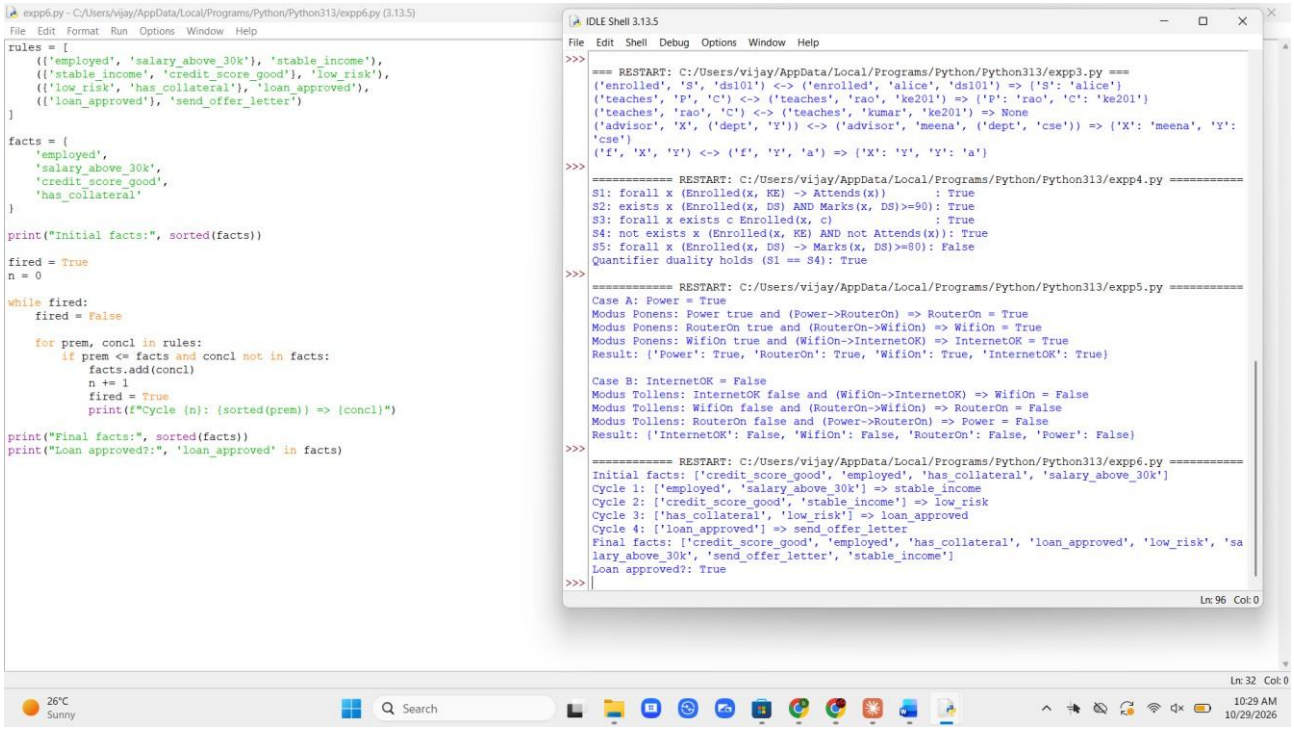
Program

```
rules = [
    ({'employed', 'salary_above_30k'}, 'stable_income'),
    ({'stable_income', 'credit_score_good'}, 'low_risk'),
    ({'low_risk', 'has_collateral'}, 'loan_approved'),
    ({'loan_approved'}, 'send_offer_letter'),
]
facts = {'employed', 'salary_above_30k', 'credit_score_good', 'has_collateral'}

print("Initial facts:", sorted(facts))
fired = True
n = 0
while fired:
    fired = False
    for prem, concl in rules:
        if prem <= facts and concl not in facts:
            facts.add(concl)
            n += 1
            fired = True
            print(f"Cycle {n}: {sorted(prem)} => {concl}")

print("Final facts:", sorted(facts))
print("Loan approved?", 'loan_approved' in facts)
```

Output Screenshot



Result

Four rules fired in sequence and the system concluded that the loan is approved and an offer letter should be sent. Forward chaining derives all consequences of the data, whether or not they were asked for.

Exercise 7: Laptop Fault Diagnosis Using Backward Chaining (Forward and Backward Chaining)

Aim

To implement backward chaining (goal-driven inference) that proves a goal by recursively proving its sub-goals.

Software Required

Python 3.x (IDLE / any Python IDE); recursion, a dictionary of rules and a set of facts

Algorithm

1. Start the program. Store the rules for battery_fault, display_fault and ram_fault in a dictionary, and the observed facts in a set.
2. To prove a goal G: if G is a known fact, return True.
3. If G is not the conclusion of any rule, return False.
4. Otherwise, for each rule that concludes G, try to prove all its premises recursively.
5. If all premises of some rule hold, G is True; otherwise G is False.
6. Print the sub-goal trace indented by depth, test all three hypotheses and stop.

Pseudocode

```
START
    rules <- battery_fault, display_fault, ram_fault
    facts <- {screen_black, power_led_off, charger_connected}
    FUNCTION prove(goal)
        IF goal IN facts THEN RETURN TRUE
        IF goal HAS NO RULE THEN RETURN FALSE
        FOR EACH rule THAT CONCLUDES goal
            IF prove(p) = TRUE FOR ALL premises p THEN RETURN TRUE
        END FOR
        RETURN FALSE
    END FUNCTION
    FOR EACH goal IN {battery_fault, display_fault, ram_fault}
        DISPLAY goal, prove(goal)
    END FOR
STOP
```

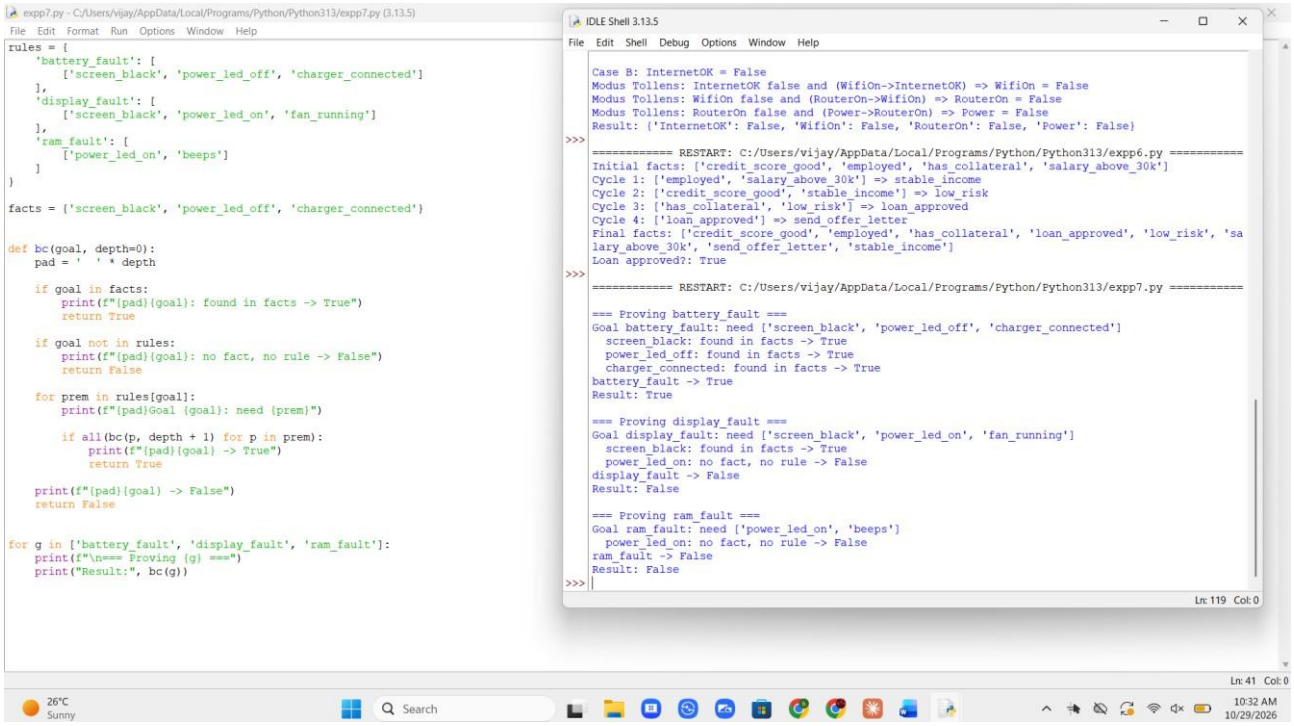
Program

```
rules = {
    'battery_fault': [['screen_black', 'power_led_off', 'charger_connected']],
    'display_fault': [['screen_black', 'power_led_on', 'fan_running']],
    'ram_fault'    : [['power_led_on', 'beeps']],
}
facts = {'screen_black', 'power_led_off', 'charger_connected'}

def bc(goal, depth=0):
    pad = '  ' * depth
    if goal in facts:
        print(f"{pad}{goal}: found in facts -> True")
        return True
    if goal not in rules:
        print(f"{pad}{goal}: no fact, no rule -> False")
        return False
    for prem in rules[goal]:
        print(f"{pad}Goal {goal}: need {prem}")
        if all(bc(p, depth + 1) for p in prem):
            print(f"{pad}{goal} -> True")
            return True
    print(f"{pad}{goal} -> False")
    return False

for g in ['battery_fault', 'display_fault', 'ram_fault']:
    print(f"\n== Proving {g} ==")
    print("Result:", bc(g))
```

Output Screenshot



Result

Only battery_fault is provable from the observations. Backward chaining examined only the sub-goals relevant to each hypothesis and stopped early on failure, which makes it efficient for diagnosis, unlike forward chaining (Exercise 6), which derives everything.

Exercise 8: Medical Screening with Bayes' Theorem (Probabilistic Reasoning)

Aim

To apply the law of total probability and Bayes' theorem for sequential belief updating in a dengue screening test.

Software Required

Python 3.x (IDLE / any Python IDE)

Algorithm

1. Start the program. Set the prior P(D) = 0.02, sensitivity = 0.95 and specificity = 0.90.
2. Compute the evidence: P(E) = P(E|D) P(D) + P(E|~D) P(~D).
3. Compute the posterior: P(D|E) = P(E|D) P(D) / P(E).
4. For a second independent test, use the first posterior as the new prior.
5. For a negative result, use P(-|D) = 1 - sensitivity and P(-|~D) = specificity.
6. Display P(Test+), the posteriors after one and two positive tests and after a negative test, then stop.

Pseudocode

```
START
    prior <- 0.02, sens <- 0.95, spec <- 0.90
    FUNCTION update(p, positive)
        IF positive THEN like_d <- sens, like_n <- 1 - spec
        ELSE like_d <- 1 - sens, like_n <- spec
        evidence <- like_d * p + like_n * (1 - p)
        RETURN (like_d * p / evidence), evidence
    END FUNCTION
    p1 <- update(prior, positive)
    p2 <- update(p1, positive)
    pn <- update(prior, negative)
    DISPLAY prior, P(Test+), p1, p2, pn
STOP
```

Program

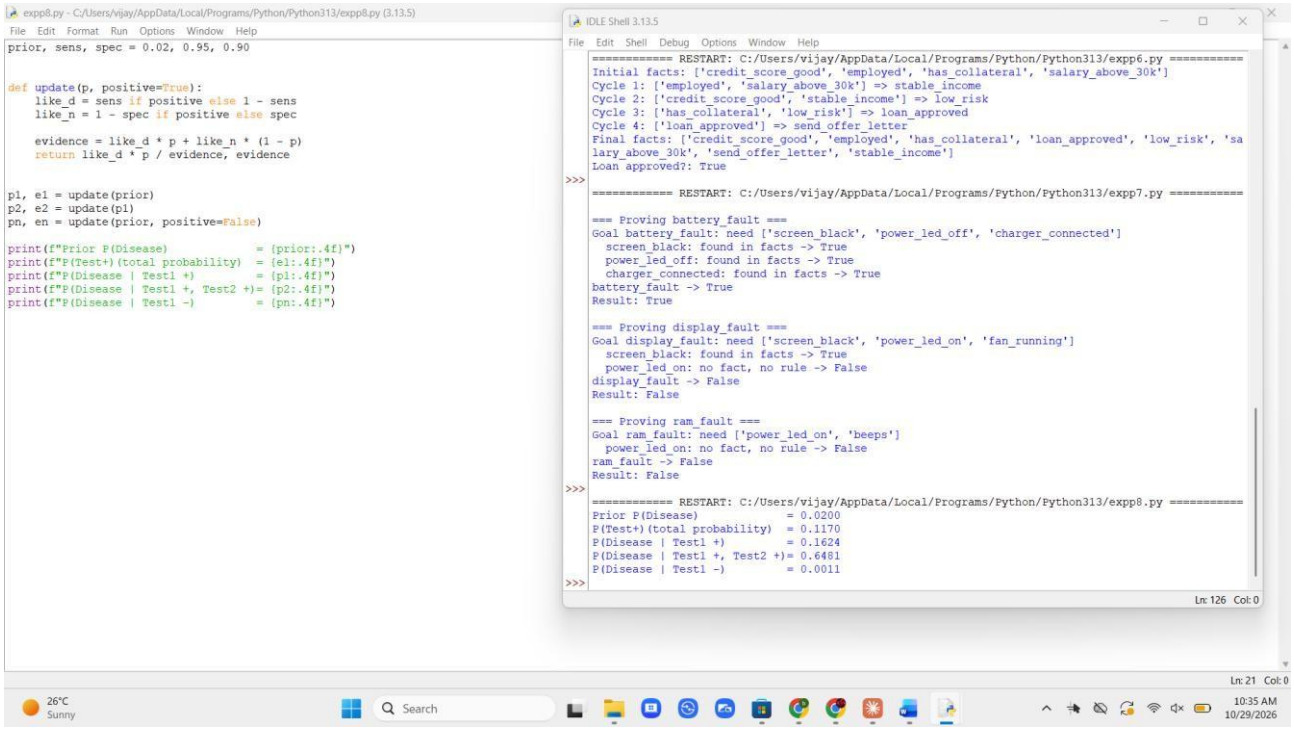
```
prior, sens, spec = 0.02, 0.95, 0.90

def update(p, positive=True):
    like_d = sens if positive else 1 - sens
    like_n = (1 - spec) if positive else spec
    evidence = like_d * p + like_n * (1 - p)
    return like_d * p / evidence, evidence

p1, e1 = update(prior)
p2, e2 = update(p1)
pn, _ = update(prior, positive=False)

print(f"Prior P(Disease)              = {prior:.4f}")
print(f"P(Test+) (total probability)  = {e1:.4f}")
print(f"P(Disease | Test1 +)          = {p1:.4f}")
print(f"P(Disease | Test1 +, Test2 +) = {p2:.4f}")
print(f"P(Disease | Test1 -)          = {pn:.4f}")
```

Output Screenshot



Result

Even with a fairly accurate test, a single positive result gives only about 16% probability of disease because the disease is rare (base-rate effect). A second independent positive raises it to about 65%. A negative result lowers it to about 0.1%.

Exercise 9: Burglar Alarm Inference Using a Bayesian Network (Bayesian Networks)

Aim

To perform exact inference in a Bayesian network by enumerating the joint distribution.

Software Required

Python 3.x (IDLE / any Python IDE) and the itertools module

Algorithm

1. Start the program. Define the CPTs: P(B), P(E), P(A|B,E), P(J|A) and P(M|A).
2. Compute the joint probability as P(B,E,A,J,M) = P(B) P(E) P(A|B,E) P(J|A) P(M|A).
3. For a query variable X with evidence e, sum the joint over all worlds consistent with X = x and e, for each value x.
4. Normalise the two sums so that they add up to 1.
5. Run the queries P(B | J, M), P(B | J), P(E | J, M) and P(A | J, M) and print the results.
6. Stop the program.

Pseudocode

```
START
    DEFINE P(B), P(E), P(A | B, E), P(J | A), P(M | A)
    FUNCTION joint(b, e, a, j, m)
        RETURN P(b) * P(e) * P(a | b, e) * P(j | a) * P(m | a)
    END FUNCTION
    FUNCTION query(var, evidence)
        FOR EACH value IN {TRUE, FALSE}
            total <- SUM OF joint(world) FOR ALL worlds
                WITH var = value AND world MATCHES evidence
        END FOR
        NORMALISE the two totals
    END FUNCTION
    FOR EACH (var, evidence) IN query list
        DISPLAY query(var, evidence)
    END FOR
STOP
```

Program

```
import itertools

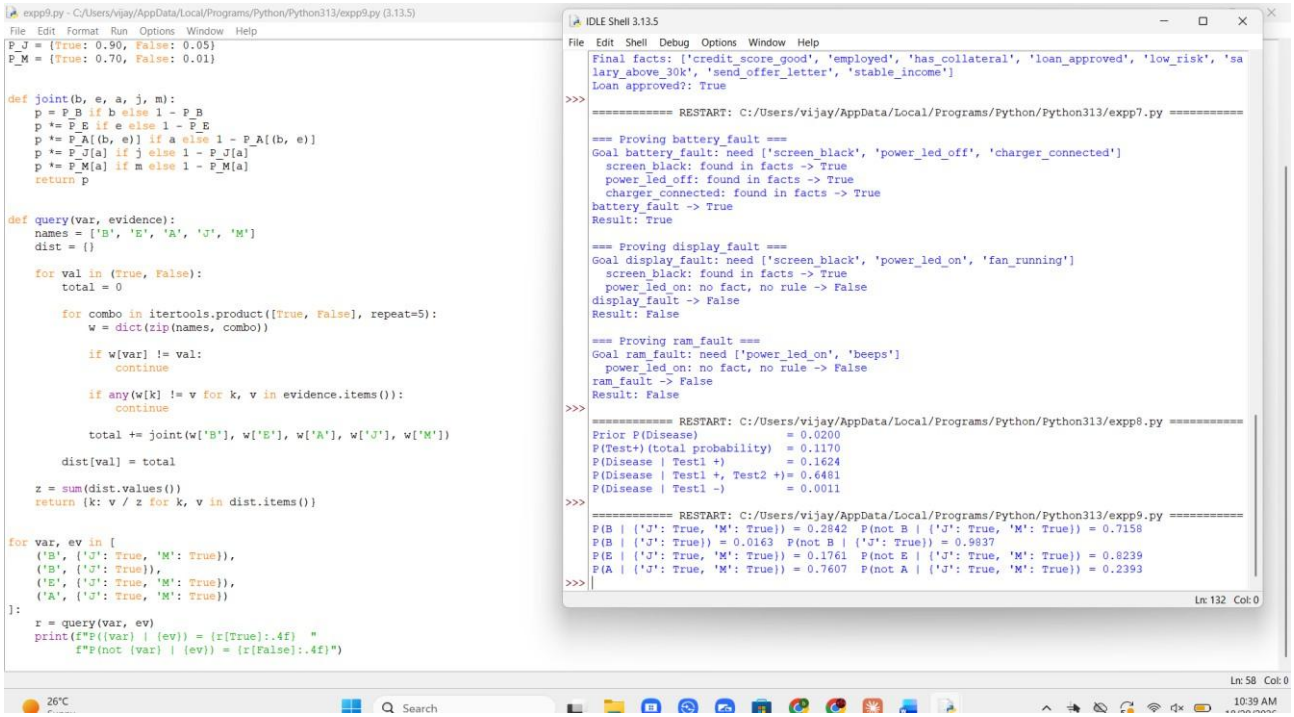
P_B, P_E = 0.001, 0.002
P_A = {(True, True): 0.95, (True, False): 0.94,
       (False, True): 0.29, (False, False): 0.001}
P_J = {True: 0.90, False: 0.05}
P_M = {True: 0.70, False: 0.01}

def joint(b, e, a, j, m):
    p = (P_B if b else 1 - P_B) * (P_E if e else 1 - P_E)
    p *= P_A[(b, e)] if a else 1 - P_A[(b, e)]
    p *= P_J[a] if j else 1 - P_J[a]
    p *= P_M[a] if m else 1 - P_M[a]
    return p

def query(var, evidence):
    names = ['B', 'E', 'A', 'J', 'M']
    dist = {}
    for val in (True, False):
        total = 0
        for combo in itertools.product([True, False], repeat=5):
            w = dict(zip(names, combo))
            if w[var] != val:
                continue
            if any(w[k] != v for k, v in evidence.items()):
                continue
            total += joint(*combo)
        dist[val] = total
    z = sum(dist.values())
    return {k: v / z for k, v in dist.items()}

cases = [('B', {'J': True, 'M': True}), ('B', {'J': True}),
         ('E', {'J': True, 'M': True}), ('A', {'J': True, 'M': True})]
for var, ev in cases:
    r = query(var, ev)
    print(f"P({var}=T | {ev}) = {r[True]:.4f}   P({var}=F | ...) = {r[False]:.4f}")
```

Output Screenshot



Result

With both neighbours calling, the probability of a burglary rises from the prior 0.1% to about 28.4%. With only John calling it is about 1.6%. The alarm itself is about 76% likely to have gone off. Evidence propagates through the network even though no single call is conclusive.

Exercise 10: Fan Speed Controller Using a Mamdani Fuzzy System (Fuzzy Logic Systems)

Aim

To design a fuzzy inference system using fuzzification, rule evaluation, aggregation and centroid defuzzification.

Software Required

Python 3.x (IDLE / any Python IDE); triangular membership functions and a discretised universe 0 to 100

Algorithm

1. Start the program. Define triangular membership functions for temperature (cold, warm, hot), humidity (dry, humid) and speed (low, medium, high).
2. Fuzzify: compute the membership degrees of the temperature and humidity inputs.
3. Rule evaluation: the firing strength of each rule is min(temperature degree, humidity degree), where AND = min.
4. Aggregation: for each output set take the max of the strengths of the rules pointing to it.
5. Clip each output set at its strength and combine the clipped sets with max.
6. Defuzzify: centroid = sum(x * mu(x)) / sum(mu(x)) for x = 0 to 100.
7. Compute the fan speed for the three input pairs, print the result and stop.

Pseudocode

```
START
    DEFINE triangular sets for temperature, humidity, speed
    DEFINE 6 rules (temperature AND humidity -> speed)
    FOR EACH (t, h) IN input pairs
        T <- memberships of t, H <- memberships of h
        FOR EACH rule
            strength[output] <- MAX(strength[output], MIN(T[temp], H[hum]))
        END FOR
        num <- 0, den <- 0
        FOR x <- 0 TO 100
            mu <- MAX over output sets OF MIN(strength, set membership at x)
            num <- num + x * mu, den <- den + mu
        END FOR
        DISPLAY speed <- num / den
    END FOR
STOP
```

Program

```
def tri(x, a, b, c):
    if x <= a or x >= c:
        return 0.0
    return (x - a) / (b - a) if x <= b else (c - x) / (c - b)

def temp_sets(t):
    return {'cold': tri(t, -1, 0, 20), 'warm': tri(t, 15, 25, 35),
            'hot': tri(t, 30, 40, 41)}

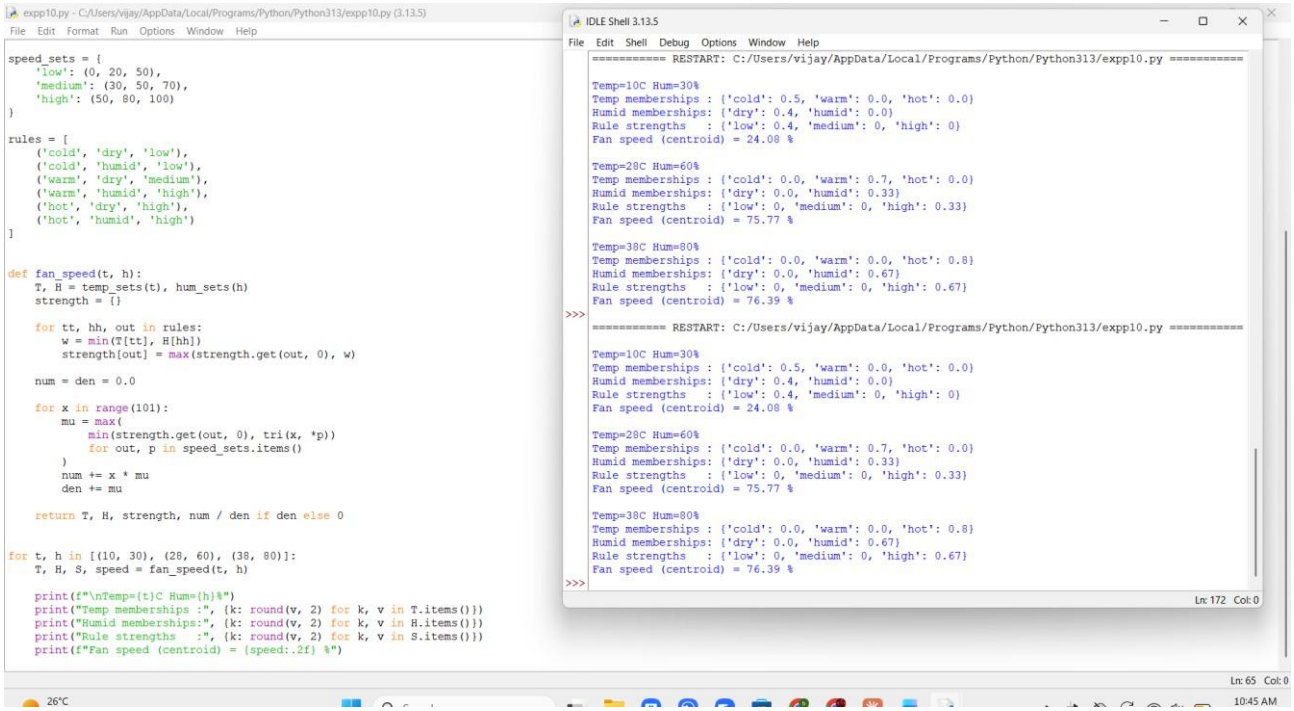
def hum_sets(h):
    return {'dry': tri(h, -1, 0, 50), 'humid': tri(h, 40, 100, 101)}

speed_sets = {'low': (0, 20, 50), 'medium': (30, 50, 70), 'high': (50, 80, 100)}
rules = [('cold', 'dry', 'low'), ('cold', 'humid', 'low'),
         ('warm', 'dry', 'medium'), ('warm', 'humid', 'high'),
         ('hot', 'dry', 'high'), ('hot', 'humid', 'high')]

def fan_speed(t, h):
    T, H = temp_sets(t), hum_sets(h)
    strength = {}
    for tt, hh, out in rules:
        w = min(T[tt], H[hh])
        strength[out] = max(strength.get(out, 0), w)
    num = den = 0.0
    for x in range(0, 101):
        mu = max(min(strength.get(o, 0), tri(x, *p))
                 for o, p in speed_sets.items())
        num += x * mu
        den += mu
    return T, H, strength, (num / den if den else 0)

for t, h in [(10, 30), (28, 60), (38, 80)]:
    T, H, s, y = fan_speed(t, h)
    print(f"Temp={t}C Hum={h}%")
    print("  temp memberships  :", {k: round(v, 2) for k, v in T.items()})
    print("  humid memberships :", {k: round(v, 2) for k, v in H.items()})
    print("  rule strengths    :", {k: round(v, 2) for k, v in s.items()})
    print(f"  Fan speed (centroid) = {y:.2f} %\n")
```

Output Screenshot



Result

The controller produced smooth, gradual outputs: about 24% for a cool, dry room, about 76% for a warm, humid room and about 76% for a hot, humid room. Rule strengths come from partial membership rather than hard thresholds, which is the main advantage of fuzzy control.